Engineering Applications of Artificial Intelligence 148 (2025) 110400

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

Binarizing historical architectural drawings with shallow convolutional autoencoders

Mark Jeremy G. Narag [a,*], Gerard Rey Lico [b], Maricor Soriano [a]

[a] National Institute of Physics, University of the Philippines Diliman, Quezon City, 1101, Philippines
[b] College of Architecture, University of the Philippines Diliman, Quezon City, 1101, Philippines

ARTICLE INFO

Keywords:
Binarization
Convolutional autoencoder
Architectural drawing
Line art

ABSTRACT

We developed convolutional autoencoder models to binarize 26 severely stained 1938 architectural drawings of the Department of Agriculture and Commerce Building in Manila. Due to the presence of deep-seated stains that have accumulated over time on the original linen drawings, traditional thresholding techniques failed in binarizing them. We propose utilizing the capabilities of deep learning models, particularly convolutional autoencoders, to automate the binarization of all 26 drawings. Our goal is to efficiently train a model using a segment from a single drawing, thereby eliminating the requirement for separate training on each individual drawing. We manually binarized a section of one drawing to serve as the desired output within our training dataset. Moreover, since deep learning models are computationally heavy, we proposed simplifying them into shallow models by trimming down the architecture to a single hidden layer. We created four pairs of models (deep and shallow) with input sizes 32×32, 64×64, 128×128, and 256×256. All the models effectively binarized the drawing by successfully separating the pronounced stains from the drawn lines, which the traditional binarization techniques have failed to do. The models achieved F1 scores from 0.961 to 0.977, intersection-over-union scores from 0.925 to 0.955, and peak-signal-to-noise ratio values from 23.1 to 25.4. Notably, we achieved higher performance metrics on shallow autoencoders than their deep counterparts. Our workflow also succeeded in binarizing a separate collection of line art drawings on vellum by another architect, which were at a lower resolution.

1. Introduction

Built heritage serve as profound symbols of a community's rich history, collective identity, and patrimony, making their conservation a matter of utmost importance. To ensure their conservation and prolonged life, adaptive reuse (AR) is a recognized conservation strategy (Pintossi et al., 2023). AR is the process of introducing a new compatible function to old structures while retrofitting their building systems to address obsolescence and ensure continued use with renewed relevance to contemporary society (FatemehHedieh et al., 2022). Not only does it retain their heritage attributes and values (Bullen and Love, 2011; Remoy, 2014), but it can also reduce the production of demolition waste (Yung and Chan, 2012) and construction costs (Bullen and Love, 2011; Yung and Chan, 2012; Alba-Rodriguez et al., 2021; Shipley et al., 2006). As such, AR is a sustainable practice as it recycles the building and the embodied carbon of the old building is conserved in situ.

In the Philippines, AR was used to transform the historical 1938 Agriculture and Commerce Building into the National Museum of Natural History. Designed by Filipino architect Antonio Toledo in 1938 under the auspices of the Bureau of Public Works, this neoclassical building was constructed in 1940, just before World War II, and was destroyed during the Battle of Manila in 1945. After the war, it was reconstructed to approximate the original plans. In 2012, the National Museum (NM) began planning the conversion of the building into the National Museum of Natural History (https://www.nationalmuseum.gov.ph/our-museums/national-museum-of-natural-history/.). NM architects retrieved 26 original architectural drawings from the Department of Public Works and Highways, which our group digitally scanned. The hand-drawn plans, stored rolled up, had accumulated dirt and stains.

These architectural drawings are crucial in the decision-making process during the conservation planning and implementation. For architects and engineers, it is essential that they are digitized and converted to vector format. However, historical documents are susceptible

* Corresponding author.
E-mail address: mnarag@nip.upd.edu.ph (M.J.G. Narag).

https://doi.org/10.1016/j.engappai.2025.110400
Received 29 September 2024; Received in revised form 7 February 2025; Accepted 22 February 2025
Available online 1 March 2025

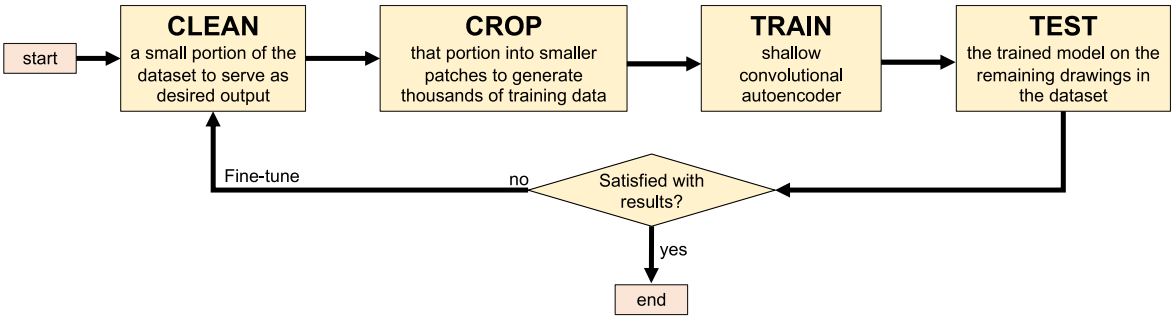

Fig. 1. Workflow to binarize architectural line art.

to degradation due to physical and chemical weathering, biodeterioration, or neglect. Stains can be as dark as the lines of the drawings, obscuring the legibility of the details and complicating direct conversion to vector format. Therefore, the architectural drawings should undergo digital "cleaning," such as image binarization.

The most basic binarization technique is thresholding. For architectural plans, saving them as binary images suffices. When an old drawing is scanned as a gray-level image, a fixed intensity threshold converts grayscale pixels below the threshold to black and those above to white. Alterations include Otsu's (Otsu, 1979) thresholding, which minimizes within-class variance and maximizes between-class variance; Niblack's (Niblack, 1985) pixel-wise thresholding using a sliding window; and Sauvola's (Sauvola and Pietik§inen, 2000) modification, which calculates the threshold using the dynamic range of the standard deviation. These traditional techniques work well on decent-quality images but often fail on degraded documents. In 2015, our group used the difference-of-Gaussians (DoG) filter for binarization, which detects edges by subtracting two Gaussian-blurred images (Masil and Soriano, 2015). However, DoG requires optimal parameters for each drawing.

Deep neural networks have shown high performance in binarizing challenging documents. The Document Image Binarization Competition (DIBCO) dataset, active from 2009 (Gatos et al., 2009) to 2019 (Zhao et al., 2019), contains both machine-printed and handwritten images scanned at 300 dots-per-inches (dpi), with ground truth binarized versions created by experts. In 2017, Tensmeyer and Martinez (Tensmeyer and Martinez, 2017) used fully convolutional neural networks for DIBCO and Palm Leaf Manuscripts (PLM) binarization. The following year, Westphal et al. (Westphal et al., 2018) employed recurrent neural networks, and Vo et al. (Vo et al., 2018) introduced a hierarchical deep supervised network. In 2019, Calvo-Zaragoza and Gallego (Calvo et al., 2019) applied a convolutional autoencoder to binarize various datasets, while Zhao et al., (2019) used a generative adversarial network (GAN) for local and global binarization of the DIBCO dataset. In 2022, Suh et al., (2022a) applied a similar model to each color channel separately, resulting in higher scores across multiple datasets.

While previous studies have excelled in binarizing degraded documents on paper or smooth backgrounds, there is a gap in research on other historical document types. These documents vary in content, mediums, resolutions, and degradation levels, unlike current public datasets. For instance, our architectural drawings are hand-drawn line art on linen, scanned at high resolutions, with rough textures and various stains. The mentioned deep models also demand significant computational resources, risk overfitting on limited data, and may struggle with non-textual content like line art. This underscores the need for a model that reduces overfitting and adapts to diverse historical documents. On the other hand, traditional techniques require tedious parameter fine-tuning for each document. Thus, we aim to develop a model that is automatic, simple, and effective, balancing the strengths of deep neural networks with the simplicity of traditional methods.

The only work closely related to ours is a 2022 preprint by Suh et al. (2022b), which binarized low resolution hand-drawn architectural floor plans from the National Archives of Korea. Their extracted 24 pixel features and trained a random forest classifier to determine pixel rendering as black or white.

Museums worldwide possess diverse historical documents that vary in degradation, medium, resolution, and content, such as line art, creating significant challenges for effective binarization. Existing models trained on datasets like DIBCO, which focus on text documents at specific resolutions, may struggle with museum collections that include non-paper mediums or higher resolution scans. To address this, we propose a novel approach for binarizing historical documents that can be tailored to the unique characteristics of museum datasets, offering a general solution for researchers rather than competing with existing deep learning models.

In this paper, we detail how we binarized the 26 severely stained architectural drawings of the Department of Agriculture and Commerce Building in Manila. These hand-drawn line art drawings, scanned at 470 dpi, were binarized using shallow convolutional autoencoder models trained on a subset of one drawing, enabling generalization to the other 25. Our method effectively separated stains from the lines despite significant degradation. We also succeeded in binarizing another line art collection from a different artist, of different medium and resolution.

2. Novelty

We introduce three novel contributions.

2.1. Application on a unique dataset

Our first contribution is the application of artificial intelligence (AI) to a unique dataset. While previous studies have used deep learning for the document binarization of handwritten and typewritten texts on paper (Westphal et al., 2018; Vo et al., 2018; Tensmeyer and Martinez, 2017; Calvo et al., 2019; Zhao et al., 2019; Suh et al., 2022a) and low-resolution architectural plans on paper (Suh et al., 2022b), our dataset consists of 26 digitized high-resolution architectural plans drawn on linen by the same architect. In contrast to prior work that typically dealt with texts on paper with smooth surfaces scanned at 300 dpi, our study binarizes line art on linen, a material with a rough surface, scanned at a higher resolution of 470 dpi. The rough surface can accentuate stains, making our dataset more challenging to binarize. Additionally, since AI models typically handle small input sizes (e.g., 256×256 pixels), we employed a local binarization approach by segmenting these high-resolution images into smaller patches. We cropped the images into portions suitable for the autoencoder, generating thousands of training samples. During testing, each patch is binarized individually, and the patches are then stitched back together to reconstruct the full high-resolution image.

2.2. Shallow convolutional autoencoder for binarization task

We propose the use of shallow convolutional autoencoders as an

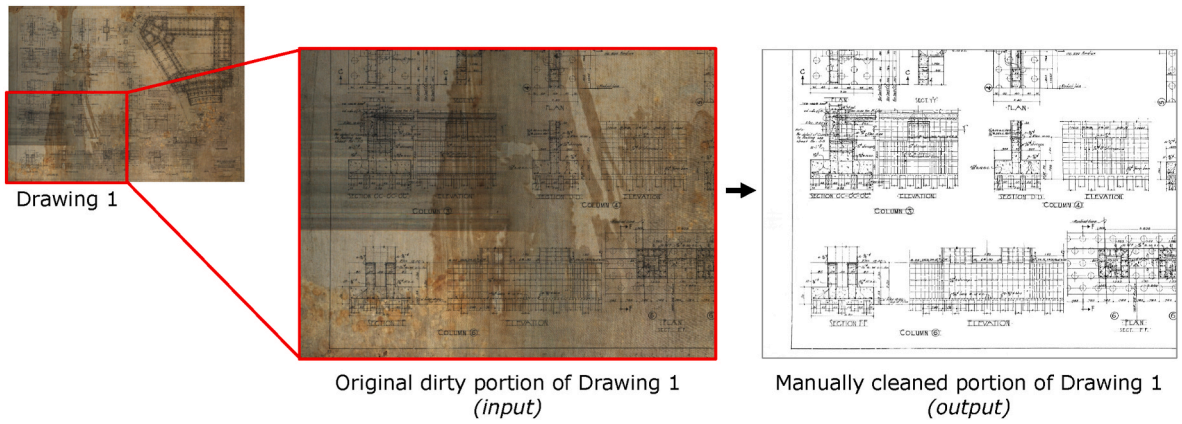

Fig. 2. Creating the desired output of the model by manually binarizing or "cleaning" a portion of Drawing 1.

alternative to traditional deep learning models. Shallow encoders retain the generalization advantages of convolutional neural networks (CNNs) while eliminating the complexity and computational demands associated with training deep models. This approach offers a more efficient and effective solution for binarization tasks, avoiding the excessive resource requirements of deep learning. Our emphasis on shallow models is motivated by the need for replicability among museums and researchers with limited computational resources. By leveraging shallow convolutional autoencoders, we provide an accessible and practical solution for users with varying technical expertise. This accessibility ensures that our approach can be implemented by institutions lacking the infrastructure for deep learning while still achieving effective results. Additionally, shallow models streamline the training process, facilitating quicker experimentation and iteration. And unlike Suh's work (Suh et al., 2022b), autoencoders do not need "hand-crafted" features but only examples of ideal binarized versions of the input.

2.3. Replicable workflow for binarizing architectural line art

Our third contribution is a replicable workflow for binarizing architectural line art collections which can be used by conservators and researchers dealing with different datasets. This workflow, as shown in Fig. 1, employs shallow convolutional autoencoders trained only on a small subset of the collection (less than 1% of the dataset). We achieve this by manually cleaning a drawing to create custom input-output pairs for supervised learning, ensuring that the binarized images meet high-quality standards. In cases where the model falls short, we fine-tune it by training on additional images, specifically targeting areas of the drawing where the model underperforms. *It should be noted that this workflow is only applicable to line art collections from the same artist, on the same medium, scanned at the same resolution.*

3. Methodology

3.1. Training data preparation

We scanned the original 26 architectural drawings at a resolution of 470 dpi, with each linen measuring approximately 100 cm $\times$ 80 cm. A single image has pixel dimensions no less than $18,000 \times 14,000$. In supervised learning, training an autoencoder model requires an input set (original dirty images) and the corresponding desired output set (binarized images). However, we only have access to the original dirty images. To address the lack of desired output, we generated our own clean and binarized versions. Initially, we employed a simple thresholding technique to binarize the dirty image. As anticipated, the resulting output still retained the underlying 'dirty' aspects, such as linen patterns and noticeable stains. Subsequently, we manually removed or cleaned the unwanted stains using GIMP software. The cleaning process took 21hours to complete. We want to clarify that this long duration is specific to our particular dataset and may not apply to all cases. Researchers working with less degraded or lower-resolution data may only need less time. In our case, the drawings were exceptionally stained and in high resolution, necessitating a more time-intensive cleaning process.

While we recognize that manual cleaning is labor-intensive, it is an important step as it ensures that the binarized images meet the quality standards expected by researchers. Additionally, you only need to manually clean it once, with the expectation that the model can already generalize the binarization process for the whole dataset.

Fig. 2 displays the final version of a clean and binarized image obtained from an initially dirty image. This final image will serve as the desired output during training. Notably, we did not clean the entire drawing but instead focused on a specific portion. We selected quadrant 3 of Drawing 1 due to its prominent stain, making it the most heavily affected part of the image. This selection aimed at training the model using numerous 'dirty-to-clean' pairs, ensuring robust generalization to the rest of the drawings. Moreover, given that we have 26 drawings in the collection of the same size and use only a quadrant of one drawing, our training data constitutes just 0.96% of the entire dataset.

Additionally, we designated another portion (quadrant 2) of drawing 1 to function as the ground-truth image during testing. This segment also underwent manual cleaning, requiring nearly the same amount of time as before. It is essential to note that this specific portion will not be included in the training dataset. Instead, it was reserved as an entirely new, previously unseen image. Testing this never-before-seen image allows us to evaluate the robustness of the trained model.

Another challenge is that artificial intelligence (AI) models typically require thousands of training images to be effective. In our case, we only have a limited number of architectural drawings—26, to be precise. However, given that these drawings are in high resolution with large pixel dimensions (approx. $18,000 \times 14,000$), we can break down a single drawing into thousands of overlapping small patches of varying pixel sizes — 32×32, 64×64, 128×128, and 256×256. This approach resolves the lack of data for training the AI model, essentially enabling local binarization. In total, we were able to extract around 22,500 training images. No data augmentation was applied, as we had already generated sufficient training data. Additionally, our training dataset adequately represents various line orientations, making flipping and rotation unnecessary.

3.2. Autoencoder models

Now, for each of the four varying patch sizes, we developed two types of autoencoder models: deep and shallow. Deep autoencoders typically consist of multiple hidden layers comprising convolution,

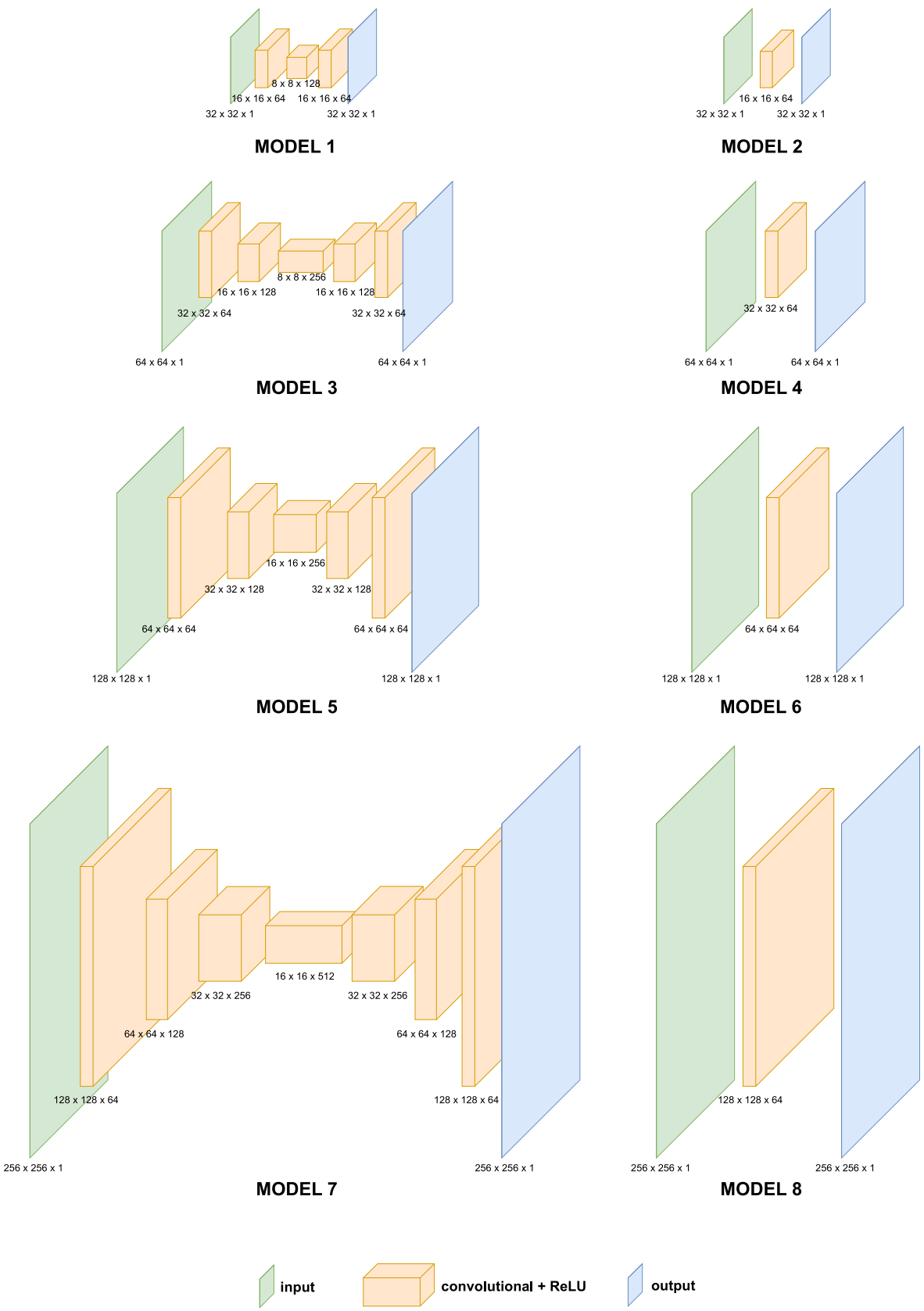

Fig. 3. Architectures of the 8 different models. Odd-numbered models are deep autoencoders while even-numbered models are shallow autoencoders. The labels $\boldsymbol{X}\times \boldsymbol{Y} \times \boldsymbol{Z}$ represents the image dimension $\boldsymbol{X} \times \boldsymbol{Y}$ and the number of filters $\boldsymbol{Z}$. Each convolution layer has a kernel size of $\mathbf{3} \times \mathbf{3}$ followed by a rectified linear unit (ReLU) operator. We chose a stride of 2 pixels during the convolution to reduce the spatial dimension into half, eliminating the need for pooling layers. The last layer has a sigmoid activation function.

deconvolution, and pooling operators. Shallow autoencoders, on the other hand, contain only a single hidden layer. In practice, deep models are often preferred over shallow ones due to their ability to extract increasingly abstract features at each layer. However, we aim to assess whether a shallow autoencoder is sufficient for our binarization task. We strive for a model that is simple yet effective.

In total, we created eight (8) distinct autoencoder models. The architectures of each model are illustrated in Fig. 3. It is important to note that the models have a single input channel, where we grayscale the image. Odd-numbered models (1, 3, 5, and 7) are deep models featuring multiple hidden layers, while even-numbered models (2, 4, 6, and 8) are shallow models with only a single hidden layer. Models 1 and 2 have an input image size of 32×32, models 3 and 4 have 64×64, models 5 and 6 have 128×128, and models 7 and 8 have 256×256.

For all models, a kernel size of 3×3 with a stride of 2 was used for each convolution and deconvolution operation, with rectified linear unit (ReLU) as the activation function. The use of a stride of 2 eliminates the need for a pooling layer, as it effectively reduces the image size after each operation, simplifying our model. We also employed the Adam solver for optimization, with a default learning rate of 0.001. In the last layer of our network, we choose a sigmoid function as our activation function to ensure that the output pixel values are within 0 (black) to 1 (white). It is defined as:

$$\sigma(x) = 1/\,(1 + exp(-x)) \tag{1}$$

where $\sigma(x)$ represents the sigmoid function and x is the input value. Sigmoid activation function has an S-shaped curve that maps any input x to a value between 0 and 1. Since the output signal is a probability-like value between 0 and 1, we applied an additional thresholding rule where it rounds values to either 0 or 1 only, ensuring that the output image is binary, with black (0) and white (1) values.

We chose a 70-30 split for training and validation data. Each model was trained for 100 epochs using either Google Colab or Google Colab Pro. Google Colab offers 12.7 GB RAM and 15 GB GPU as part of its free software, while Google Colab Pro, requiring a subscription fee per month, provides greater computational power with 83.5 GB RAM and 40 GB GPU. The models were implemented using the Keras-TensorFlow framework in Python scripts, with Python version 3.12. The computer we used was a MacBook Air M1 but since the code was executed on Google Colab's cloud service, any laptop with a web browser can be utilized to run the models. Using Google Colab also eliminates the need to install software package dependencies, streamlining the setup process.

The decision to use these two software platforms – Colab and Colab Pro, was driven by the model's complexity, which directly correlates with the computational resources required. Models 1 to 4 were executed using the free version of Google Colab, whereas models 5 to 8, with larger input image dimensions, were executed in Google Colab Pro. It is worth noting that larger input sizes generally demand more RAM for execution, hence the free version was unable to handle models 5 to 8. The training time also varies from approximately 10 minutes, for a shallow model with the smallest input size, to 75 minutes, for a deep model with the largest input size.

4. Traditional techniques

We will compare the results of our autoencoder models with traditional binarization techniques: (i) simple thresholding, (ii) Otsu's, (iii) adaptive Gaussian, and (iv) adaptive mean thresholding.

For simple thresholding, a fixed intensity value is chosen as a threshold. It categorizes pixels as black if their intensity value is below the threshold and as white if it is above. Since pixel values range from 0 to 255, this technique can yield 256 different results. We binarized the test image across all potential threshold values and determined the one yielding the highest metric score (Gonzalez and Woods, 2019).

Otsu's method, developed by Nobuyuki Otsu in 1979, is a nonparametric and unsupervised automatic threshold selection method for binarization. It aims to find the optimal threshold value by maximizing the between-class variance while minimizing the within-class variance (Otsu, 1979; Gonzalez and Woods, 2019).

One global threshold value may not be effective, especially when images have varying lighting conditions. Adaptive thresholding computes different threshold values for different image regions (Bradley and Roth, 2007). There are two common types: adaptive mean and adaptive Gaussian thresholding.

In adaptive mean thresholding, each pixel's threshold value is the mean intensity of its nearby pixels within a square neighborhood $S(x,y)$ of size $(2k+1) \times (2k+1)$, where k is half the neighborhood's size. The threshold value $T(x,y)$ for pixel (x,y) is computed as:

$$T\,(x,y) = \mu(x,y) - C \tag{2}$$

Here, $\mu(x,y)$ represents the mean intensity within $S(x,y)$, and C adjusts sensitivity to local intensity variations (Gonzalez and Woods, 2019; Dnyandeo and Nipanikar, 2016).

In adaptive Gaussian thresholding, the threshold value for each pixel (x,y) is calculated using a Gaussian-weighted sum within a window $W(x,y)$ with standard deviation σ. It is expressed as:

$$T\,(x,y) = \mu(x,y) - C \cdot \sigma(x,y) \tag{3}$$

Here, $\mu(x,y)$ is the mean intensity within $W(x,y)$, $\sigma(x,y)$ is the standard deviation, and C is a constant multiplier (Gonzalez and Woods, 2019; Dnyandeo and Nipanikar, 2016).

In adaptive thresholding, two parameters are crucial: the kernel size k or window size W, and the constant C. We varied k and W from 31, 63, 127, and 255. We aim to match the local input size of our autoencoder. C is varied from 0 to 100. These variations were implemented using OpenCV.

5. Evaluation metrics

We quantify the resulting output of all autoencoder models and traditional techniques using three metrics - F1 score, intersection-over-union (IoU), and peak signal-to-noise ratio (PSNR). F1 score, also called F-measure, is the harmonic mean of precision P and recall R. Precision measures the ability of the model not to label negative instances as positive while recall is the ability of the model to identify all positive instances. Both are obtained from the confusion matrix where each pixel is classified either black (0) or white (1). F1-score is given by

$$F1score = 2\,\mathrm{PR}/(\mathrm{P}+\mathrm{R}) \tag{4}$$

where precision and recall are denoted as follows

$$P = TP/(TP+FP) \tag{5}$$

$$R = TP/(TP+FN) \tag{6}$$

where TP, FP, and FN are denoted as true positives, false positives, and false negatives of the confusion matrix (Sasaki et al., 2007), respectively. F1 score balances precision and recall, ensuring that the binarization process accurately captures relevant features (high precision) while minimizing the omission of critical architectural details (high recall). A high F1 score indicates that the binarized image retains essential structural elements, facilitating accurate analysis and interpretation.

On the other hand, IoU is the ratio of the logical *AND* and logical *OR* of the ground truth and the output image of the model (Everingham et al., 2010). It is given by

$$IOU(f,g) = |f \cap g|/|f \cup g| \tag{7}$$

It measures the overlap between the binarized output and the ground truth, reflecting the model's ability to preserve the integrity of

Table 1
Metric scores for the eight (8) different models in comparison to the traditional thresholding techniques. Highlighted in bold is the highest F1 score, IoU, and PSNR, found in Model 6 that was executed in Google Colab Pro. We have also highlighted Model 4, which achieved the highest metric score for models executed in the free version of Google Colab. For simple thresholding, the threshold value is varied with the highest performance found in intensity 48. For Adaptive Mean and Gaussian, the kernel/window size and ***C*** parameter are varied and the highest scores are the ones presented with kernel/window size 255 for both and ***C*** parameter 19 and 17 for Adaptive Gaussian and Adaptive Mean, respectively.

Technique	F1 score	IOU	PSNR
Model 1 (deep, input 32 × 32)	0.9683	0.9385	23.99
Model 2 (shallow, input 32 × 32)	0.9745	0.9502	24.93
Model 3 (deep, input 64 × 64)	0.9612	0.9253	23.11
Model 4 (shallow, input 64 × 64)[a]	**0.9750**	**0.9511**	**25.02**
Model 5 (deep, input 128 × 128)	0.9654	0.9331	23.62
Model 6 (shallow, input 128 × 128)[b]	**0.9770**	**0.9551**	**25.40**
Model 7 (deep, input 256 × 256)	0.9616	0.9260	23.13
Model 8 (shallow, input 256 × 256)	0.9768	0.9547	25.37
Simple (threshold = 48)	0.9510	0.9083	22.07
Otsu (non-parametric)	0.3271	0.1955	5.830
Adaptive Gaussian (W = 255, C = 19)	0.8912	0.8038	18.67
Adaptive Mean (k = 255, C = 19)	0.9042	0.8252	19.18

Note: Models 1 to 4 are executed in Google Colab free version while models 5 to 8 are executed in the Pro version. Simple, Otsu, Adaptive Gaussian, and Adaptive Mean are traditional thresholding techniques.
[a] best-performing model executed in Google Colab free version.
[b] best-performing model executed in Google Colab Pro.

architectural features. A higher IoU, with 1.0 being the highest score, signifies that the binarized image closely aligns with the original, maintaining the spatial relationships crucial for architectural assessments.

Lastly, PSNR quantifies the pixel differences of the ground truth and the binarized output image of the model. Given the ground truth image f and the binarized version g of the model, both of size $M \times N$, the PSNR between f and g is given by:

$$\mathrm{PSNR}(f,g) = 10\log_{10}\left(255^2 / MSE(f,g)\right) \tag{8}$$

where

$$\mathrm{MSE}(f,g) = 1 \Big/ \mathrm{MN}\sum_{i=1}^{M}\sum_{j=1}^{N}\left(f_{ij} - g_{ij}\right)^2 \tag{9}$$

The PSNR value approaches infinity as the MSE approaches zero (Hor et al., 2010). The higher the value of the PSNR, the closer the two images in terms of pixel difference. High PSNR indicates that the binarization process has preserved the visual quality of the original document, ensuring that fine details necessary for architectural analysis are maintained.

6. Results and discussion

6.1. Testing the models

We utilize the reserved quadrant 2 of drawing 1 in Section 3.1 to test both our autoencoders and traditional binarization techniques. Note that this image was not part of the training stage and has never been seen by the models. Additionally, the manually binarized version of it will serve as the ground truth during the computation of our metrics.

Table 1 shows the F1 score, IOU, and PSNR scores for all eight autoencoder models and the traditional techniques. Since traditional techniques are evaluated with different parameter combinations, only the set of parameters that yielded the highest metric scores are presented in the table. For simple thresholding, a pixel intensity of 48 provided the highest metric score. For adaptive thresholding, the optimal kernel/window size is 255 for both types. The best values for C are 19 for Gaussian and 17 for mean.

Examining the overall picture, the table demonstrates that all eight autoencoder models have surpassed the traditional techniques, achieving higher scores across all metrics. This indicates that our approach is effective and superior to traditional techniques. Notably, in the context of utilizing shallow or deep models, it is remarkable that models with shallow architectures—models 2, 4, 6, and 8—yielded higher metric scores than their deep model counterparts. This observation proves that shallow models are already sufficient for this binarization task. Shallow autoencoder models have lesser computational complexity than deep models, emphasizing their effectiveness and efficiency. This reduced complexity in shallow models minimizes overfitting and enhances generalization across different architectural drawings. With fewer parameters, they efficiently learn the necessary features without excessive computational demands. Unlike deep networks, which are designed for hierarchical feature extraction in complex tasks like image recognition, our task primarily involves detecting repetitive and simple patterns. As a result, deep architectures are unnecessary, and shallow networks provide a more practical and effective solution, ensuring accurate binarization while maintaining efficiency in training and deployment. Moreover, the results show no signs of overfitting, as the models successfully binarized previously unseen images.

Among the four shallow models, model 6 with an input size of 128 × 128 yielded the highest score, with not much significant difference compared to the other three shallow models. It is worth noting that models 2 and 4 can be executed using the free version of Google Colab, while models 6 and 8 require higher RAM. In this case, we can conclude that models 2 and 4 are already sufficient for the task without the need to pay for extra RAM.

To provide additional visual context to our results, we present in Fig. 4 the four sample sections of the binarized outputs using Model 6, contrasting them with the optimal output achieved through traditional techniques. Clear distinctions between the techniques are evident, with our approach notably superior and closer to the ground truth. These samples address four distinct challenges encountered in binarizing heavily stained architectural drawings.

Sample 1 addresses a challenge where dark stains in the linen overpower the foreground strokes. Enclosed in red boxes are the important parts to focus on. Simple thresholding fails to separate the stains from the letters, rendering them unreadable. Both adaptive Gaussian and mean thresholding partially extract the letters from the stains. However, our approach provides a significantly clearer separation of stains and letters. Sample 2 focuses on faint pencil sketches that were not properly erased. Traditional techniques render these pencil artifacts, which is undesirable. It was overdrawn. Our approach, however, successfully distinguishes between ink pen and pencil marks.

Sample 3 depicts a pure stained background, which ideally should be all white. Traditional techniques still detect the stained patterns, whereas ours does not. Sample 4 demonstrates the challenge of thin ink strokes. Simple thresholding detects these strokes but also the stains. Both adaptive thresholding methods fail to detect thin lines, whereas ours succeeds. Overall, traditional techniques tend to overemphasize stains and pencils while underemphasizing thin lines. Our approach effectively addresses these challenges, resulting in a close approximation to the ground truth.

Finally, we evaluated our Model 6 using the remaining 25 architectural drawings. These never-before-seen images do not have ground truth counterparts for reference; therefore, quantitative metrics cannot be computed. We will rely on the qualitative results of the model moving forward.

As illustrated in the first two columns of Fig. 5, our autoencoder successfully binarized the drawings. Notably, in Drawing 10, the model managed to binarize the drawing even in the presence of heavy stains as dark as the lines. These are just a few examples where our model excelled. Of course, there are a few exceptions, which we highlighted in red boxes in Fig. 5. These portions of Drawings 10, 11, 13, and 16 are

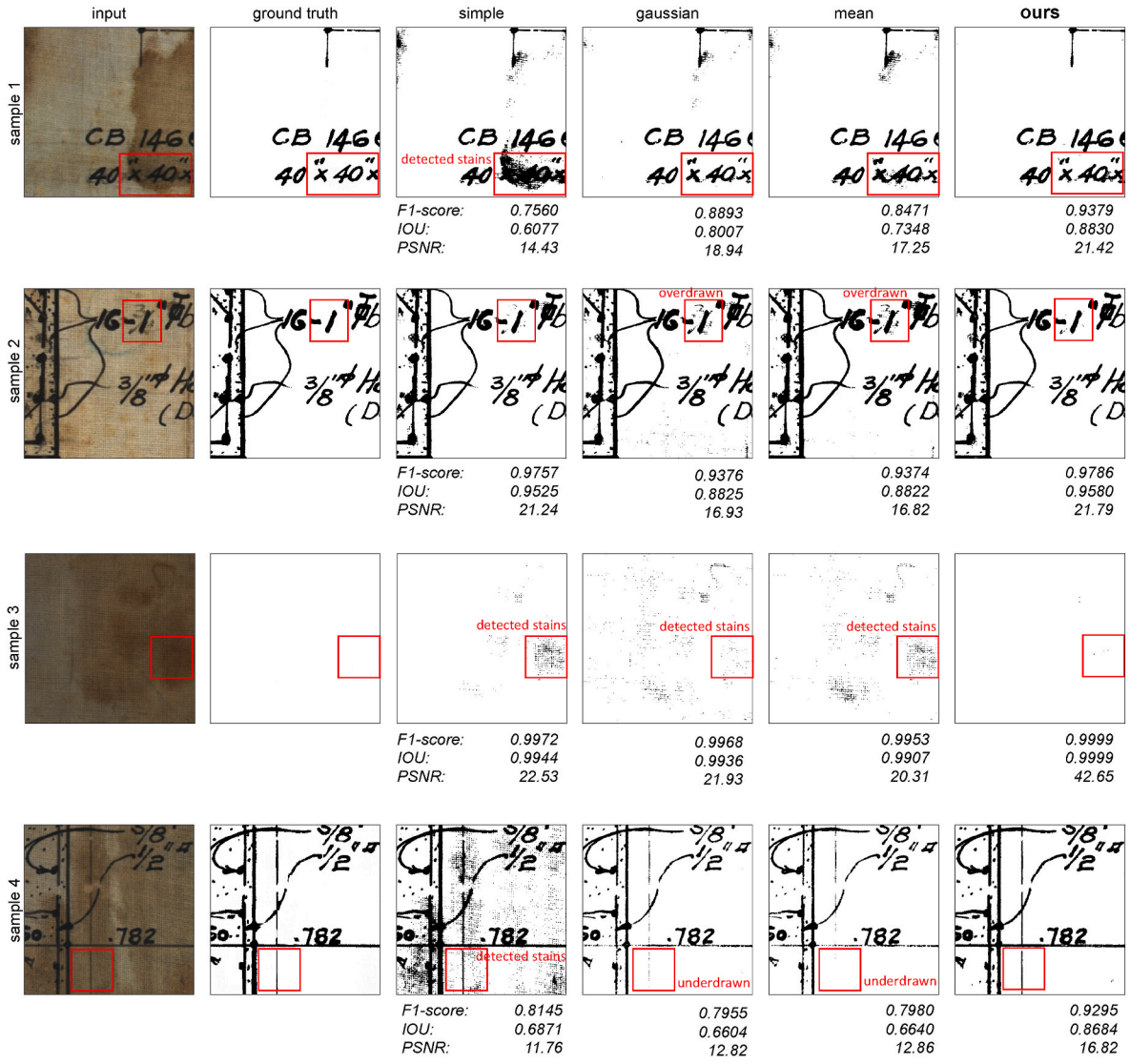

Fig. 4. Comparison of binarization outputs: traditional vs. Convolutional Autoencoder (Model 6). Each sample (**470** × **470** pixels) from quadrant 2 of Drawing 1. Red boxes highlight discrepancies from the ground truth. Samples depict challenges overcome by our model 1) Dark stains overshadowing foreground letters. 2) Overdrawn faint pencil sketches detected by traditional methods. 3) Pure background with stains. 4) Thin ink pen lines missed by traditional extraction techniques. (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

where our model encountered challenges. Faint lines, such as bricks and window details, were not detected in Drawings 11 and 13, while particularly severe stains and tape marks were not properly removed in Drawings 10 and 16. These limitations can be attributed to the training dataset used in Section 3, which did not include examples of faint lines or tape marks. To address this, we fine-tuned the model, and the improved version is shown in the last column of Fig. 5. The fine-tuning process will be detailed in the next section.

6.2. Fine-tuning autoencoder model

We fine-tuned our model by expanding the training dataset with additional patches from other drawings. Fig. 6 displays sample images added to the training dataset. Each image is limited to 1024× 1024 pixels to reduce manual cleaning time. Similar to our previous method, overlapping small patches of 128 × 128 pixels were cropped from each image to augment the training set. The additional images include various types of stains, from degrading linen to tape marks, as well as examples of thin lines.

The retrained autoencoder model 6, with the same parameters as before but with the inclusion of additional training images, showed significant improvements. The last column of Fig. 5 - “autoencoder after fine-tuning”, displays the enhanced results, where both limitations were addressed. For instance, in drawing 11, the original model failed to detect faint brick lines, but after fine-tuning, these lines were successfully detected. Similarly, in drawing 13, the design of the door was not initially detected, but after fine-tuning, a clear representation of the door’s design emerged, with improved details. In drawing 10, an improved removal of the dark stain was achieved from the fine-tuned model. In drawing 16, the tape mark, which was not present in the original training dataset, was now avoided by the model. It is important to note that these four samples were not part of the new training dataset; they were used only for evaluation purposes.

In summary, while some limitations persist, particularly regarding the inadequate detection of faint lines and severe stains, understanding the root causes of these failures allows us to address them. We hypothesize that these issues stem from the training dataset’s lack of examples featuring such characteristics. To resolve this, we initiated a fine-tuning process by expanding the dataset with additional patches that include faint lines and stains. This solution successfully improved detection in

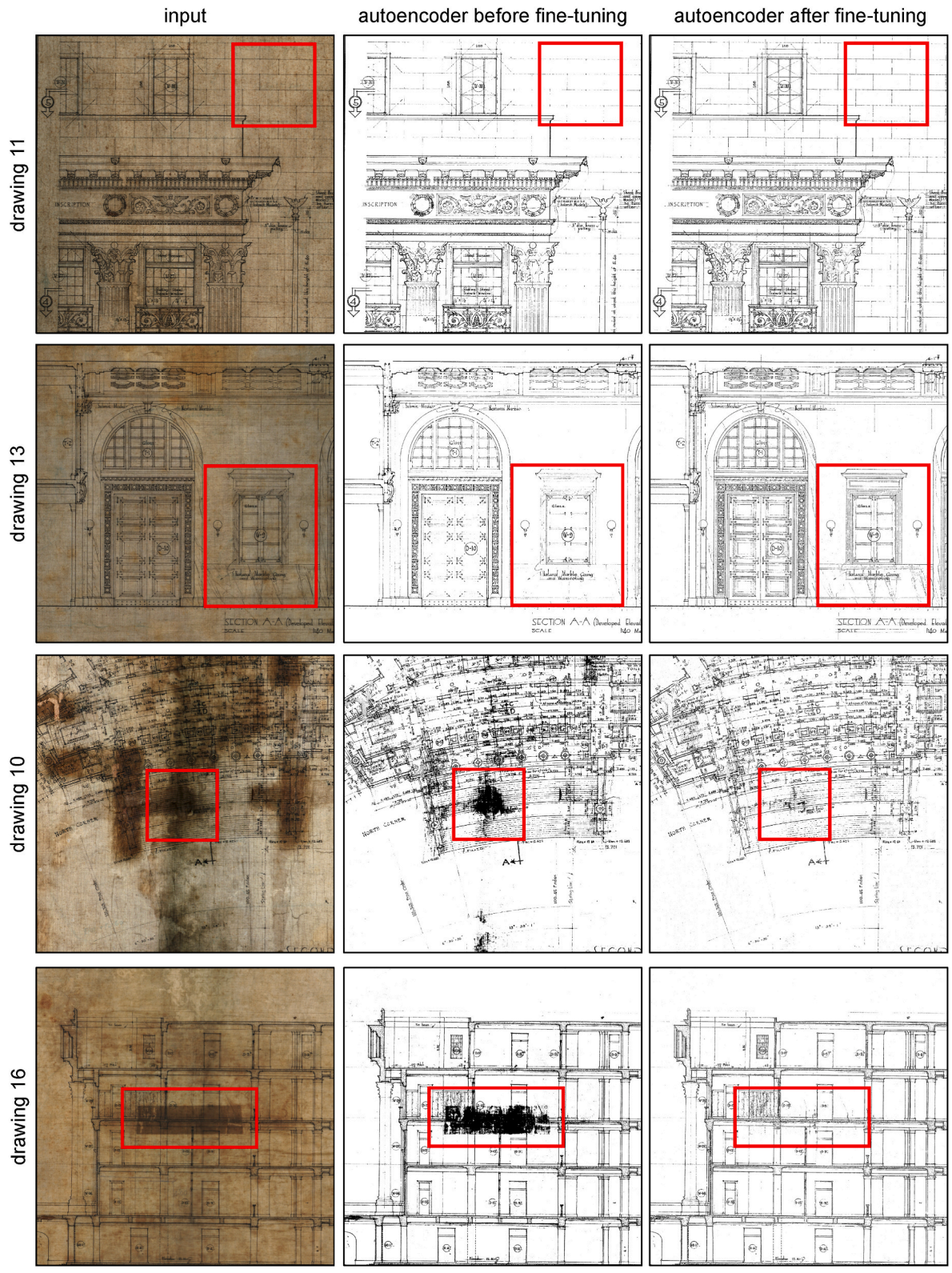

Fig. 5. Comparing our original model to the fine-tuned version, where we incorporated faint lines and other stains during re-training. The fine-tuned model successfully addressed limitations - detecting faint lines and distinguishing between tape stains and severe stains. In drawing 11, bricks were successfully detected. In drawing 13, details of the door and window are now apparent. In drawing 10, the dark stain in the middle was avoided. In drawing 16, the tape stain was avoided. Each sample size is 10 inches $\times$ 10 inches (4700 pixels $\times$ 4700 pixels).

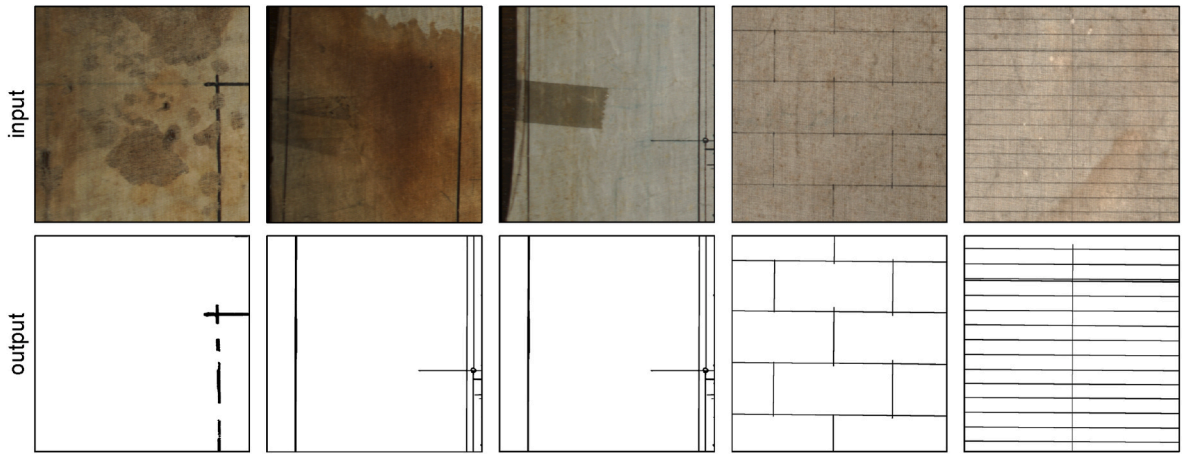

Fig. 6. Fine-tuning the autoencoder model 6 by incorporating faint lines and other stains not present in the training dataset (quadrant 3 of drawing 1). Each sample has a size of 1024 pixels × 1024 pixels. Similar to the training process, each sample was manually cleaned, and overlapping patches of size 128 pixels × 128 pixels were cropped to obtain enough training samples.

the retrained model, as evidenced in Fig. 5.

Our approach of using shallow convolutional autoencoders trained only on a subset of a single drawing, successfully binarized all 26 historical drawings that were severely stained. This approach significantly surpasses traditional thresholding techniques, which require tedious parameter optimization that cannot be universally applied across diverse drawings in the archive. Traditional methods often require users to engage in a labor-intensive process of discovering the optimal parameter set, as there is no one-size-fits-all solution for achieving the best binarization results.

Moreover, the absence of a ground truth clean version for our architectural drawings complicates the parameter optimization process, demanding manual and visual inspection for each parameter variation. This inherent challenge underscores the unique advantage of our approach. Our approach not only outperforms traditional techniques in producing superior binarization results but also operates fully automatically, thereby eliminating the need for painstaking parameter tuning. This automation enhances the efficiency of the binarization process, making it more accessible for users dealing with various historical documents. By striking a balance between the complexity of deep learning models and the simplicity of traditional methods, our approach of utilizing shallow convolutional autoencoder offers a practical solution tailored to the specific challenges posed by historical architectural drawings.

Additionally, binarizing an entire drawing with the trained model takes approximately 5 minutes, and model training never exceeds 2 hours. For the entire collection, we were able to binarize all images in just about 2 hours. In contrast, manual cleaning would require 2184 hours, or 91 days, assuming one quadrant takes 21 hours to clean, as outlined in Section 3.1. This demonstrates the significant time-saving advantage of our approach, making it a highly efficient solution for large-scale collections.

6.3. Testing on different architectural drawings

Finally, we aim to further validate whether our workflow - using shallow autoencoders trained only on a subset of the collection, applies to other databases. We used another digitized collection of architectural drawings by architect Juan Arellano, which consists of over 100 hand-drawn architectural drawings on vellum. This collection is relatively less degraded, with issues mainly related to tape marks and dirt accumulated in the creases of the medium. Additionally, the drawings are in a lower resolution of 100 dpi compared to the 470 dpi in our collection. Note that the model trained on high-resolution linen drawings will not work on these low-resolution vellum drawings. Instead, we need to train a new model specifically for this dataset by replicating our proposed workflow.

Following our workflow, we manually cleaned a single drawing from

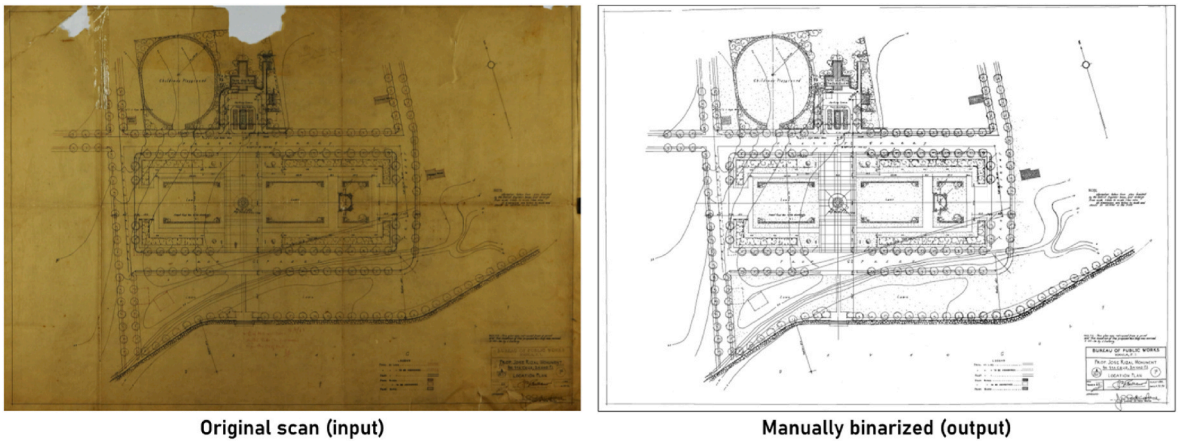

Fig. 7. Selected training pair for the Arellano collection (Proposed Jose Rizal Monument for Sta. Cruz, Davao, 1936). The same process of manual cleaning was implemented. Unlike the previous collection, which used linen as the medium, this one is in vellum. Additionally, the resolution is lower, at 100 dpi, compared to the 470 dpi of the previous collection.

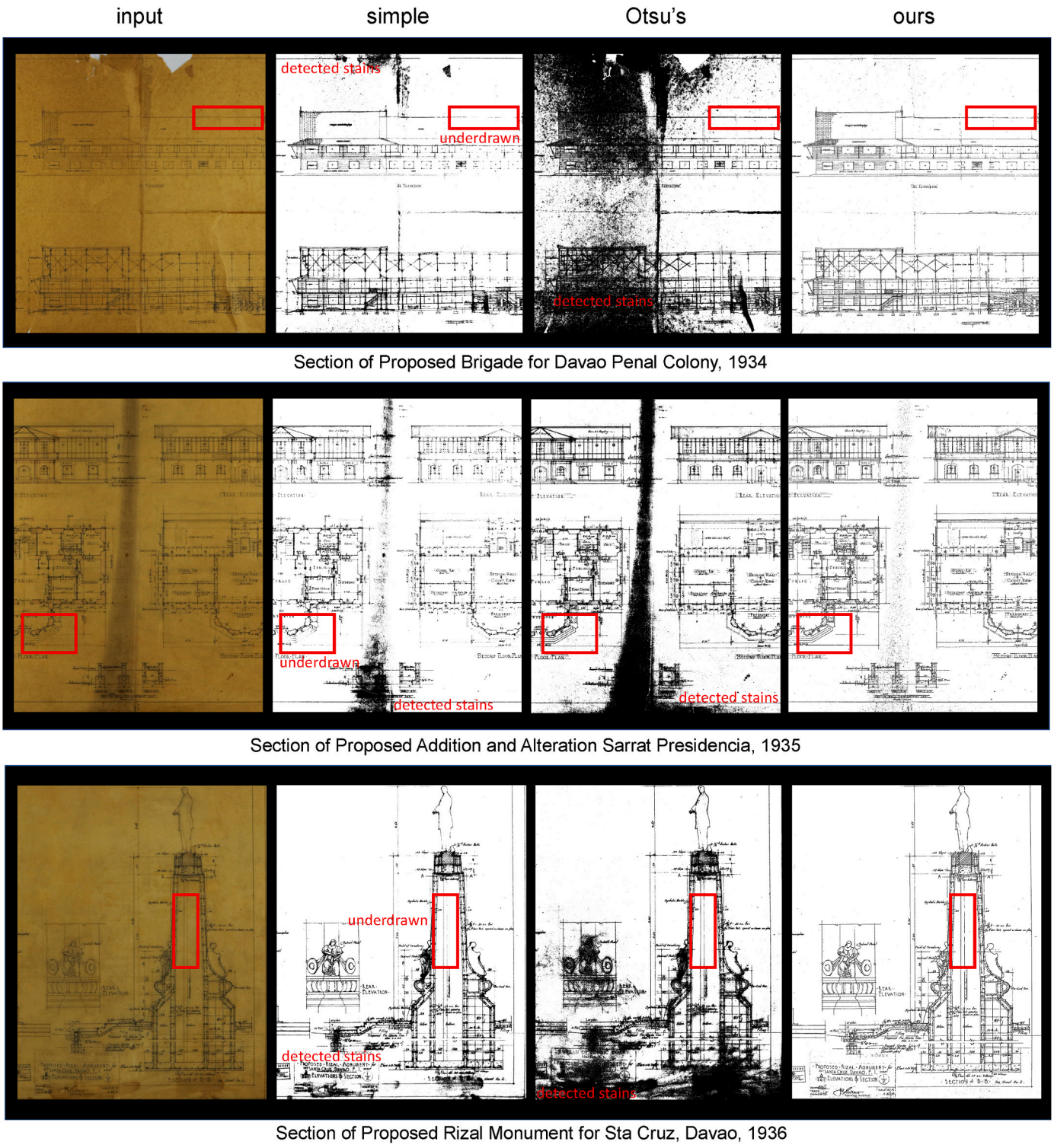

Fig. 8. Sample tests on the rest of the drawings from the Arellano collection using simple thresholding, Otsu's, and our shallow autoencoder model 4. The optimal threshold value for each drawing using a simple thresholding technique is 127 (top), 131 (middle), and 102 (bottom). Red boxes highlight faint lines in the drawings. (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

the Arellano collection, as shown in Fig. 7. Due to the intricate curves in the drawing, it took us 20 hours to complete the manual binarization. Unlike our previous database, which primarily consisted of straight lines, this new collection features irregular shapes that are more challenging to trace manually. As before, the training image was decomposed into thousands of overlapping small patches. We extracted approximately 17,000 training images for this task. We opted to use Model 4, as it performed well with our previous data and did not require a Google Colab Pro subscription.

After training on a single image from the collection, which constitutes less than 1% of the entire dataset, we tested our model on never-before-seen drawings in the collection. Our approach, as shown in Fig. 8, outperforms both simple thresholding and Otsu's method in terms of visual quality. For simple thresholding, we selected the optimal

threshold value by visually comparing the best binarization output. Notice how the different drawings have different optimal threshold values. This proves our statement that optimal parameters from one drawing cannot be universally applied to all drawings in the archive. We added Otsu's since it is non-parametric. On the other hand, we did not use Adaptive Gaussian or Adaptive Mean techniques due to the need for two parameters to tune for each drawing, which contradicts the goal of developing a general solution applicable to the entire collection. Since ground truth images are unavailable for never-before-seen images, we relied on qualitative visual assessment for evaluation.

In the three examples provided in Fig. 8, our model effectively ignores most of the stains and pronounced creases in the architectural drawings, unlike simple thresholding and Otsu's. These two methods detected the stain which obscured the actual lines. Notably, our model successfully renders faint lines, as highlighted in the red boxes. While Otsu's method also detects faint lines, it does so at the cost of retaining stains. Our model addresses both challenges - accurately rendering faint lines while removing stains. The consistent results clearly show that our approach significantly outperforms the other two techniques in terms of visual clarity. These results further validate the effectiveness of our workflow in binarizing any line art, regardless of medium, resolution, or degree of deterioration.

7. Conclusion

Using our novel approach of utilizing shallow convolutional autoencoders trained only on a subset of the collection, we successfully binarized all 26 high-resolution digitized architectural drawings of the Agriculture and Commerce Building, as well as another collection of different medium, resolution, and level of degradation.

We summarize four important findings in this study. Firstly, despite the data hunger of AI models, we demonstrated that leveraging high-resolution images enables the extraction of thousands of small patches, resolving the challenge of limited training datasets. Secondly, we achieved automatic cleaning of all drawings by training the models solely on a segment of a single drawing (less than 1% of the dataset), showcasing the robustness and efficiency of our approach. Thirdly, our trained models consistently outperformed traditional binarization techniques, as evidenced by both quantitative metrics and qualitative assessments. Notably, our non-parametric model surpassed traditional methods that require parameter optimization for each drawing, thereby enhancing overall efficiency. Lastly, our utilization of shallow convolutional autoencoders underscores their sufficiency for the binarization task. The inherent simplicity of shallow models, compared to deep counterparts, translates to faster training times, reduced RAM usage, and decreased computational complexity, making them a practical choice for binarization tasks.

There are potential limitations to our approach. One notable challenge, common to most machine learning solutions, is the availability of sufficient data. As we demonstrated, our small dataset of 26 high-resolution images was adequate because we could break them down into smaller patches, creating a larger training dataset. Yet, this approach may struggle when faced with both small and low-quality datasets. In such cases, data augmentation techniques, such as rotation and flipping could enhance the dataset's diversity and improve model robustness. Moreover, we can artificially degrade available high-quality images into low-quality versions, effectively increasing the dataset size and allowing the model to learn from a wider range of image qualities.

Another limitation is the assumption that the collections are scanned on the same media. Different media may require different trained models. Additionally, we assume that all images in the collection have the same resolution (dpi). Significant differences in resolution can lead to model failure. For instance, a drawing scanned at 470 dpi will display clear strokes within a 64 by 64 window, whereas the same drawing scanned at 100 dpi will appear almost entirely black within the same window. To address this, implementing multi-model approaches that account for different media types and resolutions could improve the adaptability of our framework. Using ensemble methods that combine predictions from multiple models trained on different datasets or configurations could further enhance accuracy and robustness.

Looking ahead, if computational constraints are no longer a concern, exploring the use of generative adversarial networks (GANs) and other state-of-the-art models may provide additional insights and improvements in the binarization of historical line art drawings. Working within these limitations, our approach represents a significant step forward in the digital archiving of historical line art documents.

CRediT authorship contribution statement

Mark Jeremy G. Narag: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Gerard Rey Lico:** Writing – review & editing, Resources, Data curation. **Maricor Soriano:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The authors thank the National Museum of the Philippines for access to the original architectural plans of the Agriculture and Commerce Building of Manila. This work was funded by the UP Intelligent Systems Center Research Grant (ISRGA2024-34).

Data availability

Data will be made available on request.

References

Alba-Rodriguez, M.D., Machete, R., Gloria Gomes, M., Paula Falcao, A., Marrero, M., 2021. Holistic model for the assessment of restoration projects of heritage housing. case studies in lisbon. Sustain. Cities Soc. 67, 102742. ISSN 2210-6707.
Bradley, D., Roth, G., 2007. Adaptive thresholding using the integral image. J. Graph. Tool. 12, 13.
Bullen, P.A., Love, P.E., 2011. Adaptive reuse of heritage buildings. Struct. Surv. 29, 411.
Calvo-Zaragoza, J., Gallego, A.-J., 2019. A selectional auto-encoder approach for document image bi- narization. Pattern Recogn. 86, 37.
Dnyandeo, S., Nipanikar, R., 2016. A review of adaptive thresholding techniques for vehicle number plate recognition. Int. J. Adv. Res. Comput. Commun. Eng. 5, 944.
Everingham, M., Van Gool, L., Williams, C.K., Winn, J., Zisserman, A., 2010. The pascal visual object classes (voc) challenge. Int. J. Comput. Vis. 88, 303.
Fatemeh Hedieh Arfa, B.L., Zijlstra, Hielkje, Quist, W., 2022. Adaptive reuse of heritage buildings: from a literature review to a model of practice. Hist. Environ.: Policy & Practice 13, 148 arXiv:
Gatos, B., Ntirogiannis, K., Pratikakis, I., 2009. Document image binarization contest (DIBCO 2009). In: 2009 10th International Conference on Document Analysis and Recognition. IEEE, pp. 1375–1382.
Gonzalez, R.C., Woods, R.E., 2019. Digital Image Processing. Pearson.
Hore, A., Ziou, D., 2010. Image quality metrics: psnr vs. ssim. In: 2010 20th International Conference on Pattern Recognition, pp. 2366–2369.
https://www.nationalmuseum.gov.ph/our-museums/national-museum-of-natural-history/.
Masil, B.E., Soriano, M.N., 2015. Line drawing enhancement of historical architectural plan using difference-of-Gaussians filter. In: 2015 International Conference on Humanoid, Nanotechnology, Information Technology,Communication and Control, Environment and Management (HNICEM), pp. 1–5.
Niblack, W., 1985. An Introduction to Digital Image Processing. Strandberg Publishing Company.
Otsu, N., 1979. A threshold selection method from gray-level histograms. IEEE Transactions on Systems, Man, and Cybernetics 9, 62.
Pintossi, N., Ikiz Kaya, D., van Wesemael, P., Pereira Roders, A., 2023. Challenges of cultural heritage adaptive reuse: a stakeholders-based comparative study in three European cities. Habitat Int. 136, 102807. ISSN 0197-3975.

Remoy, H., 2014. In: Sustainable Building Adaptation: Innovations in Decision-Making. Blackwell, pp. 159–182.
Sasaki, Y., et al., 2007. The truth of the f-measure. Teach tutor mater 1, 1.
Sauvola, J., Pietik§inen, M., 2000. Adaptive document image binarization. Pattern Recogn. 33, 225. ISSN 0031-3203.
Shipley, R., Utz, S., Parsons, M., 2006. Does adaptive reuse pay? a study of the business of building renovation in Ontario, Canada. Int. J. Herit. Stud. 12, 505.
Suh, S., Kim, J., Lukowicz, P., Lee, Y.O., 2022a. Two-stage generative adversarial networks for binariza- tion of color document images. Pattern Recogn. 130, 108810. ISSN 0031-3203.
Suh, H., Kim, H., Yu, K., 2022b. Machine learning-based binarization technique of hand-drawn floor plans.
Tensmeyer, C., Martinez, T., 2017. Document image binarization with fully convolutional neural net- works. In: 2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR), vol. 1. IEEE, pp. 99–104.
Vo, Q.N., Kim, S.H., Yang, H.J., Lee, G., 2018. Binarization of degraded document images based on hierarchical deep supervised network. Pattern Recogn. 74, 568.
Westphal, F., Lavesson, N., Grahn, H., 2018. Document image binarization using recurrent neural networks. In: 2018 13th IAPR International Workshop on Document Analysis Systems (DAS). IEEE, pp. 263–268.
Yung, E.H., Chan, E.H., 2012. Implementation challenges to the adaptive reuse of heritage buildings: towards the goals of sustainable, low carbon cities. Habitat Int. 36, 352. ISSN 0197-397.
Zhao, J., Shi, C., Jia, F., Wang, Y., Xiao, B., 2019. Document image binarization with cascaded generators of conditional generative adversarial networks. Pattern Recogn. 96, 106968. ISSN 0031- 3203.